Binary Classification of AOI Alarms into True Defects and False Calls

Second check of AOI results with supervised machine learning (Siemens production data)

P2: core code and baseline simulation · test plan · report

Yu Han · Melisa Subasi · Meron Hagos · Alejandro Rivas

Team 1 · Guided by Prof. Dr.-Ing. Hamidreza Heidari · 3 November 2026 · github.com/bodhi584/aoi-false-call-classification

Yu Han

Where the project stands

P1	20 Oct	Proposal, tech stack, MVP (revised topic 23 Oct)	**done**
P2	3 Nov	Core code + baseline simulation; test plan; report	**this phase**
P3	24 Nov	Feature progress; unit tests and CI; issues board	**next**
P4	8 Dec	Feature progress; full evaluation	**planned**
P5	22 Dec	Demo video, run instructions, packaging	**planned**
Final	12 Jan	Final presentation and report	**planned**

P2 in short

Literature (36 sources): a second check with machine learning removes false calls on a test set, but the one study that looked at later production days [8] misses 5–15 % of true defects there.

[8] names two causes – drift and an unstable threshold – and two next steps; the literature has methods for both. Those become the two variables of our simulation.

Data: the public Siemens set we use is the same as in [8]; the share of true defects changes over the weeks.

Code: data, classifier + threshold, day-by-day simulation, tests and CI run in the repository.

First result: [8] reproduced one-to-one; a correctly set threshold and retraining both lower the missed defects, but none reaches 1 % yet.

Test plan extended; P1 plan revised: we build on [8] instead of beside it.

Next (P3): the simulation software with retraining and threshold as settings.

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Melisa Subasi

AOI: what it is, how it works, why false calls matter now

≈ 99 %

of the solder joints that AOI flags are false calls [8]

Every flagged board is checked again by a person. Siemens line, 132 days, 440,274 flagged joints [5].

1 · What AOI is and where it is used [2]

In electronics assembly (SMT), a camera system inspects every solder joint after soldering: it measures the joint and compares the measurements with fixed limits. (Abu Ebayyeh & Mousavi 2020, review)

2 · How it works on a real line, and the problem [5], [8]

If one measurement is outside its limit, the board goes to a manual inspection station; an operator decides: true defect or false call. Almost all alarms are false calls, so operators mostly check good boards. (Pfab et al. 2024, data article; Pfab & Rothering 2025)

3 · Why it matters now [4]

Siemens is adding a second inspection step that decides whether an AOI alarm is a real defect or a false call; target 40 % fewer false calls, a target, not a result. (Siemens press release, Oct 2026)

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Melisa Subasi

Second check with machine learning: three studies, one baseline

Study	Data	Classifiers	Tested on later periods?	Code public?
[6] Thielen et al. 2020	own, 1,144 / 4,264 components, not public	kNN, neural network, random forest	no; threshold tuned on the evaluation data	no
[7] Albayari et al. 2026	Siemens AG; not stated whether the public set	XGBoost, RF, DNN, kNN	no; several train–test splits	no
[8] Pfab & Rothering 2025	Siemens, public [5]	kNN, RF, balanced RF, XGBoost, AutoML	yes: 5 later slices, 5–15 % missed defects	yes, with run logs

What all three show

A classifier on AOI measurements recognises false calls on a test set from the same period; random forests are among the best models [6], [8].

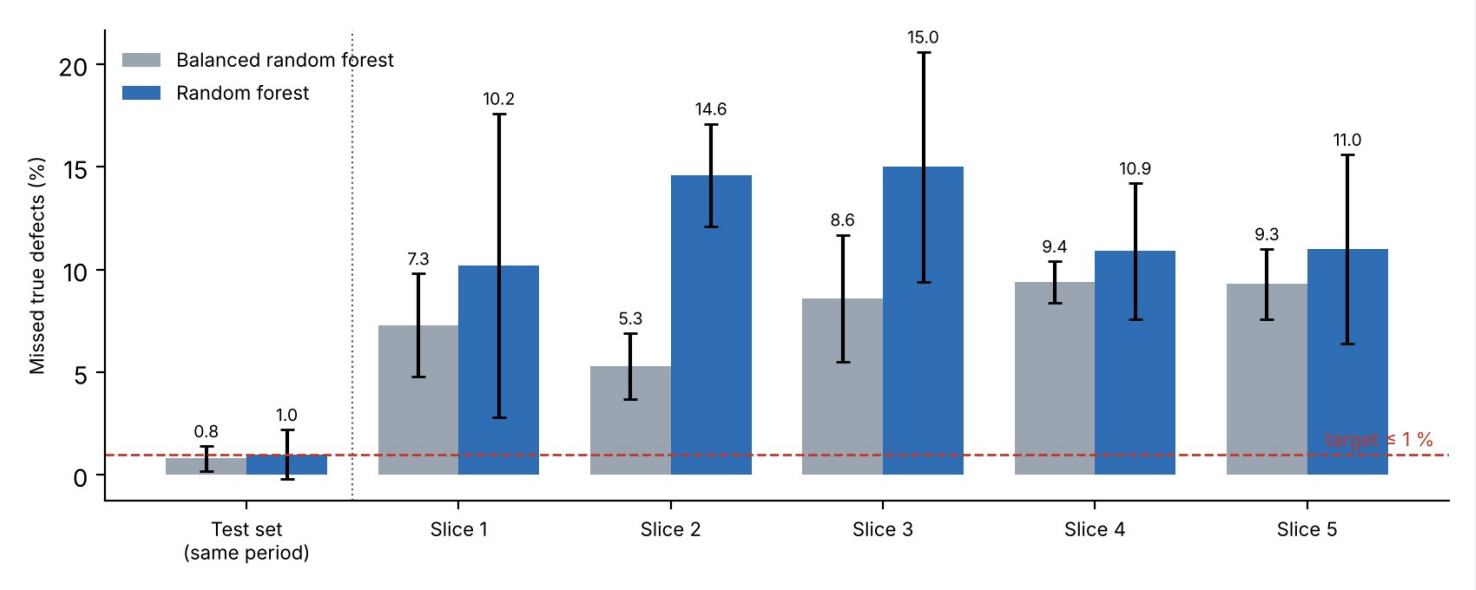
Why [8] is our core paper

Same public data as ours, published code and run logs, and the only study that tests later production periods. We can repeat it exactly and build on it.

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Melisa Subasi

[8] in detail: works on the test set, fails on later production days



Two causes named in [8]

The model was trained once, on the first half of the data. On a test set from the same period it meets the target. On the five later periods it misses 5–15 % of the true defects.

Cause 1 – drift: the production line changes over time, so a model trained on old data stops fitting [21]–[23]. The same happens in welding, chip making and sensor data [24]–[26].

Cause 2 – threshold: the cut-off between “defect” and “false call” was set once on old data; [8] itself calls this unstable, and [28] explains why it cannot guarantee the 1 % limit.

[8] names both as possible next steps and tests neither.

TECHNISCHE HOCHSCHULE DEGGENDORF

Meron Hagos

Chain of evidence

Fails over time

train once → 5–15 % missed defects later [8]

Two causes

drift [21]–[27] and an unstable threshold method [8], [28]

Methods exist

threshold: [28]–[30]; retraining: [22], [27], [31]–[33]

Works elsewhere

fraud detection, <1 % positives [35]; semiconductor materials line [36]

Open for AOI

named by [8] as next steps, not tested for the second check of AOI results

Pfab & Rothering (2025), Sec. VII: "… potential next steps to stabilize the modeling by improving the method for setting the threshold and handling the distribution drifts within the dataset."

Our simulation focuses on exactly these two steps: the threshold method and drift (retraining).

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Meron Hagos

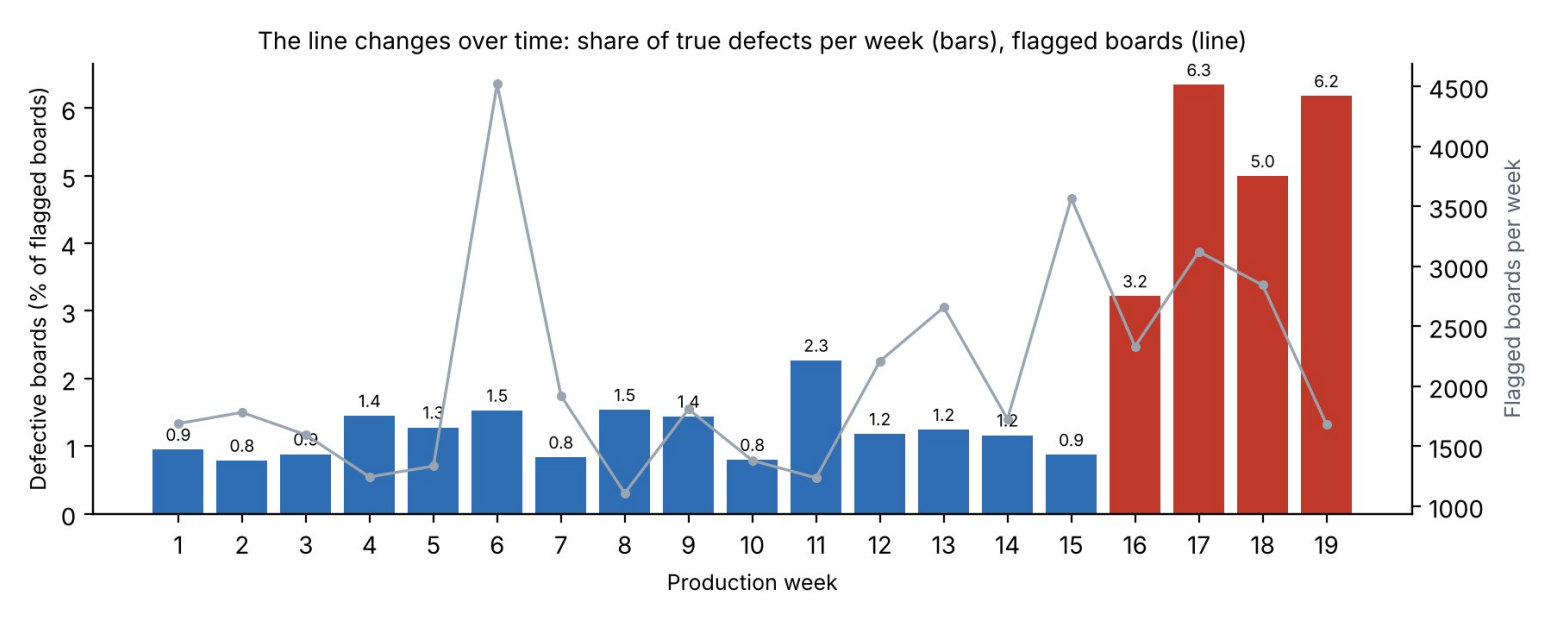
The two next steps of [8] become the variables of our simulation

Variable	Option	What it does	Sources
Threshold	as in [8]	set once on the training data, as [8] did	[8]
	with a safety margin	set so that the 1 % limit holds with 95 % confidence; needs about 300 known true defects	[28]
	re-set for the current period	adjust the cut-off when the share of true defects changes, even before new labels arrive	[29], [30]
Retraining	off	train once, as in [8]	[8]
	fixed schedule	retrain every period / week / day on all data so far, or on a recent window	[22], [31], [35], [36]
	triggered by drift	retrain only when the error rate starts to rise	[32], [33]

Every option comes from a published method; the simulation shows its effect on missed defects and released false calls next to [8].

Meron Hagos

The line changes over time



Facts (our analysis)

440,274 flagged joints, 39,742 boards, 132 days

97.8 % of flagged boards are false calls

Share of defective boards: 0.8–2.3 % per week, then 3.2–6.3 % in the last 4 weeks

Drift test: early vs. late days separable, AUROC 0.974

Alejandro Rivas

Input → simulation → output

INPUT

Recorded Siemens production

AOI measurements of each flagged solder joint, in time order [5]

published models and settings of [8]

VARIABLES

What operation would change

Retraining: off / on, frequency (per slice → weekly → daily)

Threshold: as in [8] / set correctly / set on recent days

OUTPUT

Per production period

Missed true defects (%)

False calls released (%) = manual checks saved

next to the result of [8]

Reference lines from [8]: ≤ 1 % missed true defects, ≥ 40 % false calls released.
Later (P3–P5): a software with these variables as settings and charts of both outputs.

Alejandro Rivas

Replication of Pfab & Rothering (2025): one factor at a time

	(a) Paper exactly	(b) Train once, threshold re-estimated	(c) Retrain
Data split	same as [8]	same	same
Models, hyper-parameters, seeds	RF, balanced RF; logged values; 10 seeds	same	same
Threshold	value logged in [8]	their method, applied to the final model	same method, new data
Training	once	once	before each of the 5 later slices
Purpose	reproduce [8]	(a) vs (b) = effect of the threshold	(b) vs (c) = effect of retraining

Counted per solder joint, as in [8]: missed true defects (%) and false calls released (%). We assume the manual check labels every joint, so retraining always has fresh labels (best case).

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Alejandro Rivas

What runs in the repository

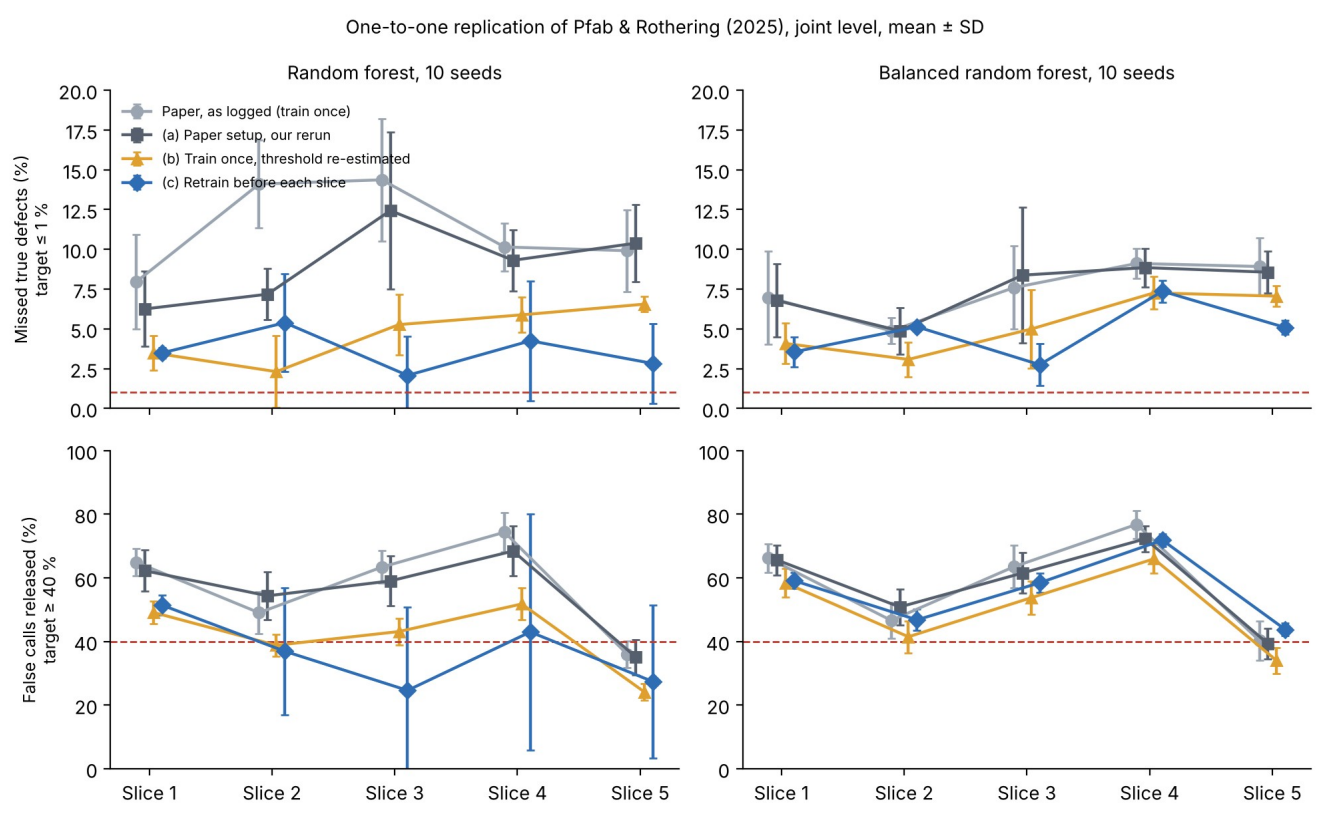
Part	File	What it does
Data	aoi_classification/data.py	load the Siemens data, boards, time order
Classifier + threshold	aoi_classification/model.py	train, score, threshold for 99 % recall
Baseline simulation	aoi_classification/simulation.py	day by day: train on earlier days only
Replication	scripts/replicate_pfab2025.py	one-to-one repeat of [8], train once vs. retrain
Classifier screening	scripts/compare_classifiers.py	13 classifiers, 8 families, same split
Tests + CI	tests/, GitHub Actions	10 tests on Windows, macOS, Linux

Tools: Python, pandas, scikit-learn, imbalanced-learn, matplotlib, Jupyter · Data: Mendeley (CC BY-NC 3.0), not stored in the repo

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Alejandro Rivas

[8] reproduced; a correct threshold and retraining both help



Reproduction ✓

Our rerun of [8] gives the same numbers: 7.5 % missed defects with balanced RF.

Retraining: small gain

Balanced RF: 5.3 → 4.8 % missed, 50.7 → 56.0 % released.

RF: 4.7 → 3.6 % missed, but fewer released.

Threshold matters most

Only fixing the threshold: 7.5–9.1 % → 4.7–5.3 % missed.

Nothing reaches the 1 % line yet — that is the work for P3.

Yu Han

How we check the code and the numbers

Group	Examples	Status
Data (D1–D7)	labels 0/1, no missing values, time order, checksum	4 exist, rest P3
Classifier + threshold (C1–C7)	threshold rule, all models run, same seed = same result	4 exist, rest P3
Simulation (S1–S4)	model only knows earlier days	1 exists, rest P3
Replication (R1–R4)	same split as [8]; (a) reproduces [8]; retraining uses only earlier data; 10 seeds	new
KPIs (K1–K5)	missed defects ≤ 1 %, false calls released ≥ 40 %; every number from results/	open

Unit tests run on every push (CI, three operating systems). A KPI that is not reached is a result, not a failed test.

Yu Han

Revision of the P1 proposal after finding the core paper [8]

	What [8] already did	P1 plan (23 Oct)	Revised plan (P2)
Second check	classifier after AOI on the Siemens data; on a same-period test set ≤ 1 % missed defects	build our own second check	use the check of [8] as baseline (one-to-one)
Over time	evaluated on 5 later slices with a static model: 5–15 % missed defects	day-by-day simulation in time order	day-by-day simulation of operation on top of [8]: the part they did not do
Causes	names drift and the threshold method; tests neither	not known yet	address both: retraining against drift; a correctly set threshold
Models	random forest, balanced random forest (also kNN, XGBoost, AutoML)	logistic regression, random forest, neural network	same as [8], so results are comparable; model choice matters little [13]
Comparison	targets ≤ 1 % missed, ≥ 40 % false calls released	recall ≥ 99 %, specificity ≥ 20 % (board level)	same metrics as [8]; compared with their later-slice results

Unchanged: Siemens AOI data [5] · input → system → output · tools (Python, scikit-learn) · milestones

TECHNISCHE HOCHSCHULE DEGGENDORF

Yu Han

Towards P3 (24 Nov)

Simulation software

one screen: retraining and threshold as settings; charts of missed defects and false calls released next to [8]

Retraining frequency

daily, every 3 days, weekly, per slice: how far do missed defects drop? [36]

Threshold with confidence

order statistic with enough left-out true defects [28]; re-set for the current period [29], [30]

More model families

gradient boosting, neural network, logistic regression in the same setup

References (1/4) — each is summarised in the bibliography report

[1] F. Psarommatis, G. May, P.-A. Dreyfus, and D. Kiritsis, "Zero defect manufacturing: state-of-the-art review, shortcomings and future directions in research," Int. J. Production Research, vol. 58, no. 1, pp. 1–17, 2020, doi: 10.1080/00207543.2019.1605228. open

[2] A. A. R. M. Abu Ebayyeh and A. Mousavi, "A review and analysis of automatic optical inspection and quality monitoring methods in electronics industry," IEEE Access, vol. 8, pp. 183192–183271, 2020, doi: 10.1109/ACCESS.2020.3029127. open

[3] P. Martinek, I. M. Putrama, O. Krammer, and A. Geczy, "Short review on machine learning optimization methods in surface mounted electronics assembly technologies," in Proc. 46th Int. Spring Seminar on Electronics Technology (ISSE), 2023, pp. 1–6, doi: 10.1109/ISSE57496.2023.10168524. open

[4] Siemens, "Siemens and NVIDIA advance AI-powered PCB inspection from Digital Twin to factory," press release, Plano, TX, USA, Oct. 21, 2026. [Online]. Available: https://news.siemens.com/en-us/siemens-nvidia-pcba-digital-twin-factory/ open

[5] K. Pfab, R. Eichler, A. Mallandur, and M. Rothering, "Data of automated optical inspection of surface-mounted technology electronic production," Data in Brief, vol. 53, Art. no. 110110, Apr. 2024, doi: 10.1016/j.dib.2024.110110. Data: Mendeley Data, v2, doi: 10.17632/99jzmh9658.2. open

[6] N. Thielen, D. Werner, K. Schmidt, R. Seidel, A. Reinhardt, and J. Franke, "A machine learning based approach to detect false calls in SMT manufacturing," in Proc. 43rd Int. Spring Seminar on Electronics Technology (ISSE), Demanovska Valley, Slovakia, 2020, pp. 1–6, doi: 10.1109/ISSE49702.2020.9121044. open

[7] J. Albayari, P. Patra, D. Patel, and D. Santos, "Enhancing false call detection in automated optical inspection using XGBoost classifier and XAI algorithms," in Flexible Automation and Intelligent Manufacturing (FAIM 2025), Lecture Notes in Mechanical Engineering. Cham, Switzerland: Springer, 2026, pp. 509–530, doi: 10.1007/978-3-032-05610-8_51. open

[8] K. Pfab and M. Rothering, "Towards improved research methodologies for industrial AI: a case study of false call reduction," arXiv:2506.14521 [cs.LG], Jun. 2025. Source code and run logs: https://gitlab.com/research9841542/FCR_1. open

[9] M. Fernandez-Delgado, E. Cernadas, S. Barro, and D. Amorim, "Do we need hundreds of classifiers to solve real world classification problems?" J. Machine Learning Research, vol. 15, no. 90, pp. 3133–3181, 2014. open

TECHNISCHE HOCHSCHULE DEGGENDORF THD

References (2/4) — each is summarised in the bibliography report

[10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track, 2022. open

[11] R. Shwartz-Ziv and A. Armon, "Tabular data: deep learning is not all you need," Information Fusion, vol. 81, pp. 84–90, 2022, doi: 10.1016/j.inffus.2021.11.011. open

[12] V. Borisov, T. Leemann, K. Sessler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: a survey," IEEE Trans. Neural Networks and Learning Systems, vol. 35, no. 6, pp. 7499–7519, 2024, doi: 10.1109/TNNLS.2022.3229161. open

[13] D. McElfresh, S. Khandagale, J. Valverde, V. Prasad C, G. Ramakrishnan, M. Goldblum, and C. White, "When do neural nets outperform boosted trees on tabular data?" in Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track, 2023. open

[14] N. Hollmann, S. Muller, L. Purucker, A. Krishnakumar, M. Korfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," Nature, vol. 637, no. 8045, pp. 319–326, 2025, doi: 10.1038/s41586-024-08328-6. open

[15] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324. open

[16] C. Chen, A. Liaw, and L. Breiman, "Using random forest to learn imbalanced data," Dept. of Statistics, University of California, Berkeley, Tech. Rep. 666, 2004. open

[17] T. Chen and C. Guestrin, "XGBoost: a scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785. open

[18] F. Pedregosa et al., "Scikit-learn: machine learning in Python," J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011. open

TECHNISCHE HOCHSCHULE DEGGENDORF THD

References (3/4) — each is summarised in the bibliography report

[19] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Trans. Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263–1284, Sep. 2009, doi: 10.1109/TKDE.2008.239. open

[20] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, Jun. 2006, doi: 10.1016/j.patrec.2005.10.010. open

[21] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodriguez, N. V. Chawla, and F. Herrera, "A unifying view on dataset shift in classification," Pattern Recognition, vol. 45, no. 1, pp. 521–530, 2012, doi: 10.1016/j.patcog.2011.06.019. open

[22] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACM Computing Surveys, vol. 46, no. 4, Art. no. 44, pp. 1–37, 2014, doi: 10.1145/2523813. open

[23] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: a review," IEEE Trans. Knowledge and Data Engineering, vol. 31, no. 12, pp. 2346–2363, Dec. 2019, doi: 10.1109/TKDE.2018.2876857. open

[24] J. C. Bauer, S. Trattnig, F. Vieltorf, and R. Daub, "Handling data drift in deep learning-based quality monitoring: evaluating calibration methods using the example of friction stir welding," J. Intelligent Manufacturing, vol. 37, no. 2, pp. 759–774, 2025, doi: 10.1007/s10845-025-02569-6. open

[25] J. Shim, S. Cho, E. Kum, and S. Jeong, "Adaptive fault detection framework for recipe transition in semiconductor manufacturing," Computers & Industrial Engineering, vol. 161, Art. no. 107632, 2021, doi: 10.1016/j.cie.2021.107632. open

[26] N. Jourdan, S. Sen, E. J. Husom, E. Garcia-Ceja, T. Biegel, and J. Metternich, "On the reliability of machine learning applications in manufacturing environments," in NeurIPS 2021 Workshop on Distribution Shifts, 2021, arXiv:2112.06986. open

[27] P. Kadlec, R. Grbic, and B. Gabrys, "Review of adaptation mechanisms for data-driven soft sensors," Computers & Chemical Engineering, vol. 35, no. 1, pp. 1–24, 2011, doi: 10.1016/j.compchemeng.2010.07.034. open

TECHNISCHE HOCHSCHULE DEGGENDORF THD

References (4/4) — each is summarised in the bibliography report

[28] X. Tong, Y. Feng, and J. J. Li, "Neyman-Pearson classification algorithms and NP receiver operating characteristics," Science Advances, vol. 4, no. 2, Art. no. eaao1659, 2018, doi: 10.1126/sciadv.aao1659. open

[29] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure," Neural Computation, vol. 14, no. 1, pp. 21–41, 2002, doi: 10.1162/089976602753284446. open

[30] Z. C. Lipton, Y.-X. Wang, and A. Smola, "Detecting and correcting for label shift with black box predictors," in Proc. 35th Int. Conf. Machine Learning (ICML), PMLR vol. 80, 2018, pp. 3122–3130. open

[31] G. Widmer and M. Kubat, "Learning in the presence of concept drift and hidden contexts," Machine Learning, vol. 23, pp. 69–101, 1996, doi: 10.1007/BF00116900. open

[32] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in Advances in Artificial Intelligence – SBIA 2004, Lecture Notes in Computer Science, vol. 3171. Berlin, Germany: Springer, 2004, pp. 286–295, doi: 10.1007/978-3-540-28645-5_29. open

[33] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in Proc. 7th SIAM Int. Conf. Data Mining (SDM), 2007, pp. 443–448, doi: 10.1137/1.9781611972771.42. open

[34] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: an analysis and review," Int. J. Forecasting, vol. 16, no. 4, pp. 437–450, 2000, doi: 10.1016/S0169-2070(00)00065-0. open

[35] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: a realistic modeling and a novel learning strategy," IEEE Trans. Neural Networks and Learning Systems, vol. 29, no. 8, pp. 3784–3797, 2018, doi: 10.1109/TNNLS.2017.2736643. open

[36] M. Gao, J. M. Perathoner, A. L. Bonin, S. Eulig, and G. Klesse, "Robust and reliable AI for predictive quality in semiconductor materials manufacturing with MLOps and uncertainty quantification," arXiv:2605.07752 [cs.LG], 2026. open

TECHNISCHE HOCHSCHULE DEGGENDORF THD

Repository

github.com/bodhi584/aoi-false-call-classification

Thank you

Team 1 · Yu Han · Melisa Subasi · Meron Hagos · Alejandro Rivas